New run

Validated, persisted as queued, then executed outside the request. The browser never calls the model.

Challenge adapter

demo-adapter (example)

Input file

Choose File No file chosen

Types: .json .jsonl .csv .txt · max size set by NFR bounds

Sample N (optional)

all records

Idempotency key

auto-generated

Dry run (validate and plan, no model calls)

INPUT_INVALID · Row 14 is missing required field "id". Fix the file and retry. (Example error state)

Start run Reset

Before you start

- Double submit returns the existing run (DUPLICATE_RUN).
- Reloading a run page never creates a new run.
- Max single-run time 900 s.
- Keys stay server-side.

Readiness

Config valid	Passed
PostgreSQL	Passed
LLM provider key	Blocked
Redis (T2)	Not run

A blocked check returns 503 with a safe message.

run_cm8x2k

Running · Sample data · design mockup

Cancel run

Waiting (provider) · LLM_RATE_LIMITED, retry 1 of MAX_TRANSIENT_RETRIES in 8 s.

Timeline

Real events by sequence ID. Reconnect replays after last seq.

- #1 validate_input — Passed · 0.2 s
- #2 gather_context — Passed · 1.4 s
- #3 reason (LLM) — Running
- #4 validate_output — Not run
- #5 evaluate — Not run

No percentage bar: only completed stages are shown.

Output | Evidence | Metrics

Output

Shown only after validate_output passes. Until then: "No output yet".

```
{ "id": "rec-001", "decision": "...", "evidence": ["doc-4#p2"] }
```

Schema valid · 1 warning

Artifacts

output.jsonl
sha256 9f2c…a1 · 48 KB

Download

API verifies the artifact belongs to this run.

System architecture

Core first, adapters second. PostgreSQL owns state; Redis only accelerates.

1 · Adapters

CLI · pnpm agentforge run, eval, validate, report, diff, preflight **T0 P0**

Web · Next.js UI + Route Handlers, SSE or polling **T1**

2 · Guard

Zod validation · rate limit · idempotency · persist before enqueue → JobRunner (inline default, BullMQ in T2)

3 · Core packages

orchestrator
State machine, retries, cancel, final status

agents + adapter-kit
Task roles, ChallengeAdapter contract

policies
Allowlist, retry, abstain, escalate

llm
One provider, structured output, usage

schemas (Zod)
Validate every untrusted boundary

evaluation
Metrics, failure clusters, baseline

PostgreSQL
Runs, stages, ordered events, artifact metadata, idempotency, usage

Artifact store
Local FS: key, size, SHA-256. T0 uses runs/<runId>/

Redis **T2 P1**
Queue, cache, limits. Flushable, never authoritative.

Tiers

T0 core + CLI → T1 web + DB → T2 worker + Redis → T3 code profile (sandbox). Next tier starts only after the previous gate passes.

How it works: states, errors, timeline

One state vocabulary. Final status is computed from evidence, never from model text.

Run state flow

Queued → Running ⇄ Waiting (provider) / Repairing (T3) → Succeeded | Succeeded with warnings | Failed | Blocked | Timed out | Cancelling → Cancelled

Terminal states never return to Running. Finalization is one guarded UPDATE, so exactly one writer wins.

Error → status

Category	Effect
`LLM_AUTH_FAILED, BUDGET_EXCEEDED`	Run blocked
`INPUT_INVALID, SCHEMA_INVALID`	Record failed, batch continues
`LLM_RATE_LIMITED, LLM_TIMEOUT`	Backoff, then failed or blocked
`LLM_INVALID_OUTPUT`	One correction, then failed
`OUTPUT_INVALID, EVALUATION_FAILED`	Run failed, not published
`DUPLICATE_RUN`	Existing run returned

Request lifecycle

1. POST /api/runs validates, persists queued
2. JobRunner enqueues by runId
3. Orchestrator runs stages, writes events
4. Output validated, then evaluated
5. Status computed, finalized once
6. UI reads events after last seq

Build timeline

11–17 Oct
T0 core, CLI, eval harness

18–21 Oct
T1 web, DB, run viewer

22–23 Oct
Mock challenge, freeze

24 Oct 6 PM IST
24 h: adapter only, eval, preflight, submit